

Time Complexity and Insertion Sort

Aimal Rextin

Department of Computing

Outline

- Time Complexity: Continued
- Sorting
- Insertion Sort
- Analysis

Time Complexity

Time complexity is basically just *counting* the number of instructions

```
for(int i=0;i<n;i++)  
{  
    cout<<"Hello";  
}
```

```
for(int i=0;i<n;i++)  
{  
  cout<<"Hello";  
}
```

$$T(n) = 3n + 2$$

$$\Theta(n)$$

Example 2

```
for(int i=0;i<n;i++)  
{  
    cout<<"Hello";  
}  
for(int j=0;j<n;j++)  
{  
    cout<<"Bye"  
}
```

$$T(n) = 3n + 2 + 3n + 2 = 6n + 4$$

$$\Theta(n)$$

Example 2

```
for(int i=0;i<n;i++)  
{  
    cout<<"Hello";  
}  
for(int j=0;j<n;j++)  
{  
    cout<<"Bye"  
}
```

$$T(n) = 3n + 2 + 3n + 2 = 6n + 4$$

$$\Theta(n)$$

Example 3

```
for(int i=0;i<1000;i++)  
{  
    cout<<"Hello";  
}
```

Example 3

```
for(int i=0;i<1000;i++)  
{  
    cout<<"Hello";  
}
```

Constant $T(n)$

$$T(n) = 3001$$

$$\Theta(1)$$

Example 4

```
for(int i=n;i>=1;i--)  
{  
    cout<<"Hello";  
}
```

Example 4

```
for(int i=n;i>=1;i--)  
{  
    cout<<"Hello";  
}
```

$$T(n) = 3n + 2$$

$$\Theta(n)$$

Example 5

```
for(int i=n;i>1;i=i/2)
{
    cout<<"Hello";
}
```

Assume that $n = 2^k$, where k is an integer

Example 5

```
for(int i=n;i>1;i=i/2)
{
    cout<<"Hello";
}
```

Assume that $n = 2^k$, where k is an integer

$$T(n) = 3 \log_2 n + 2$$

Example 6

```
for(int i=1;i < n;i=i*2)
{
    cout<<"Hello";
}
```

Assume that $n = 2^k$, where k is an integer

Example 6

```
for(int i=1;i < n;i=i*2)
{
    cout<<"Hello";
}
```

Assume that $n = 2^k$, where k is an integer

$$T(n) = 3 \log_2 n + 2$$

Example 7

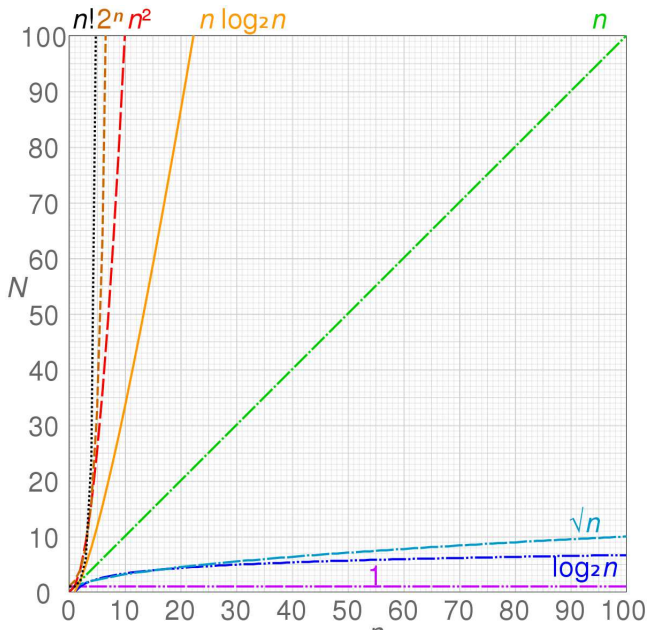
```
for(int i=1;i<= n;i=i++)  
{  
  for(int j=1;j<=n;j++)  
    cout<<"Hello";  
}
```

Example 7

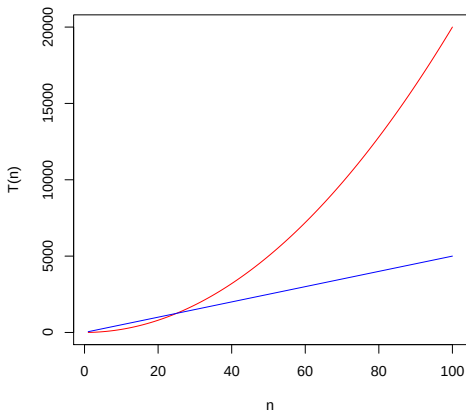
```
for(int i=1;i<= n;i=i++)  
{  
  for(int j=1;j<=n;j++)  
    cout<<"Hello";  
}
```

$$T(n) = 3n^2 + 4n + 1$$
$$\Theta(n^2)$$

Time Complexity



Larger and Smaller Time



Algo 1 $T(n) = 50x + 1$ (blue) Algo 2 $T(n) = 2x^2 + 1$ (red)

There is a point after which the quadratic function takes more time. No matter what the constants so quadratic takes more time

Order of Growth

- We care only about the *order of growth* or *rate of growth* of $T(n)$

Order of Growth

- We care only about the *order of growth* or *rate of growth* of $T(n)$
 - ▶ so we ignore lower-order terms

E.g., in

$$T(n) = an^2 + bn + c$$

we only consider the n^2 term and say that $T(n)$ is a quadratic function in n

Order of Growth

- We care only about the *order of growth* or *rate of growth* of $T(n)$
 - ▶ so we ignore lower-order terms

E.g., in

$$T(n) = an^2 + bn + c$$

we only consider the n^2 term and say that $T(n)$ is a quadratic function in n

We write

$$T(n) = \Theta(n^2)$$

and say that “ $T(n)$ is theta of n -squared”

Worst-Case Complexity

- In general we measure the complexity of an algorithm *in the worst case*

Insertion Sort

Sorting

- **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Sorting

■ **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: a sequence $\langle b_1, b_2, \dots, b_n \rangle$ such that

- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is a *permutation* of $\langle a_1, a_2, \dots, a_n \rangle$

Sorting

■ **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: a sequence $\langle b_1, b_2, \dots, b_n \rangle$ such that

- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is a *permutation* of $\langle a_1, a_2, \dots, a_n \rangle$
- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is *sorted*

$$b_1 \leq b_2 \leq \dots \leq b_n$$

■ **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: a sequence $\langle b_1, b_2, \dots, b_n \rangle$ such that

- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is a *permutation* of $\langle a_1, a_2, \dots, a_n \rangle$
- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is *sorted*

$$b_1 \leq b_2 \leq \dots \leq b_n$$

■ Typically, A is implemented as an array

Sorting

■ **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: a sequence $\langle b_1, b_2, \dots, b_n \rangle$ such that

- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is a *permutation* of $\langle a_1, a_2, \dots, a_n \rangle$
- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is *sorted*

$$b_1 \leq b_2 \leq \dots \leq b_n$$

■ Typically, A is implemented as an array

$A =$	6	8	3	2	7	6	11	5	9	4
-------	---	---	---	---	---	---	----	---	---	---

Sorting

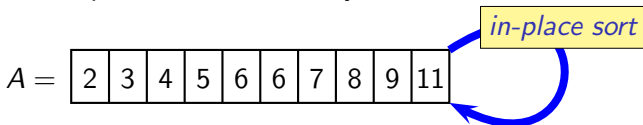
■ **Input:** a sequence $A = \langle a_1, a_2, \dots, a_n \rangle$

Output: a sequence $\langle b_1, b_2, \dots, b_n \rangle$ such that

- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is a *permutation* of $\langle a_1, a_2, \dots, a_n \rangle$
- ▶ $\langle b_1, b_2, \dots, b_n \rangle$ is *sorted*

$$b_1 \leq b_2 \leq \dots \leq b_n$$

■ Typically, A is implemented as an array



Types of Sorting

- There are two types of sorting
- **In-Place Sorting** is an sorting algorithm that use a constant extra memory space, i.e. a fixed number of additional variables
- **Out-Place Sorting** a sorting algorithm that uses an additional data structures whose size depends on the the size of the input array. This additional memory can be of the form of another array of size n for example.

Insertion Sort

- **Idea:** it is like sorting a hand of cards

Insertion Sort

- **Idea:** it is like sorting a hand of cards
 - ▶ scan the sequence left to right
 - ▶ pick the value at the current position a_j
 - ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$

Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$

$$A = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 6 & 8 & 3 & 2 & 7 & 6 & 11 & 5 & 9 & 4 \\ \hline \end{array}$$

Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

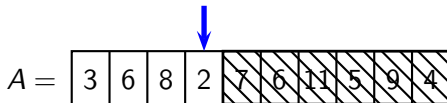
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

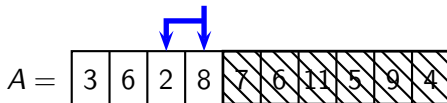
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

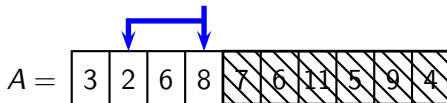
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

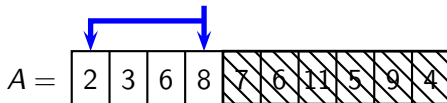
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

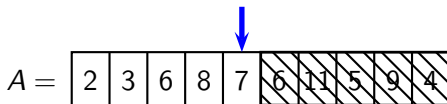
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

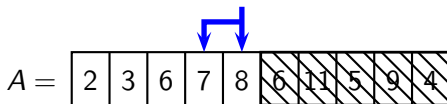
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

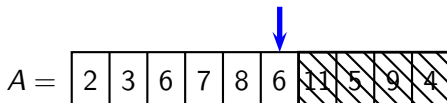
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

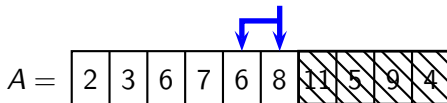
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

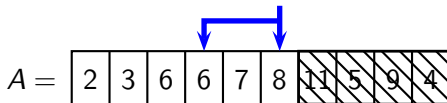
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

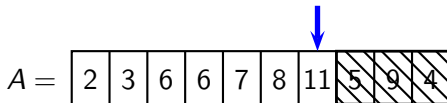
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

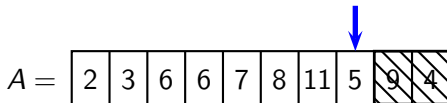
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

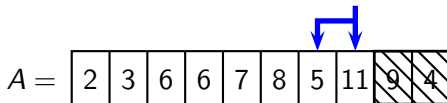
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

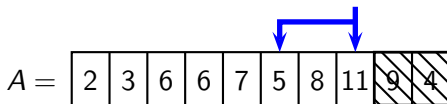
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

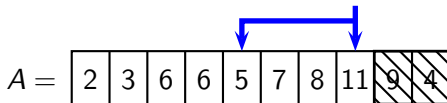
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

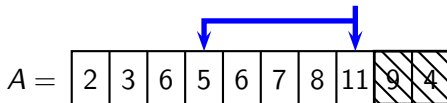
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

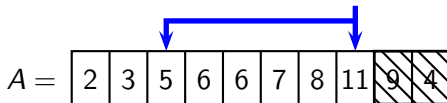
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

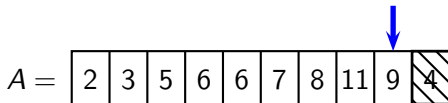
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

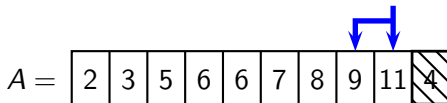
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

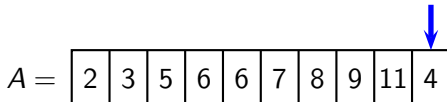
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

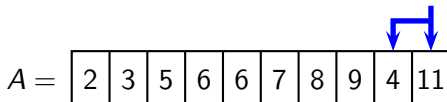
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

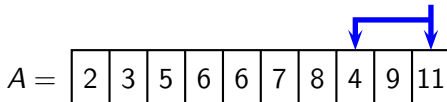
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

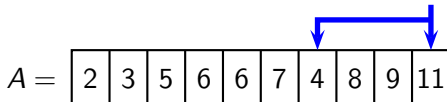
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

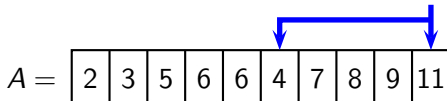
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

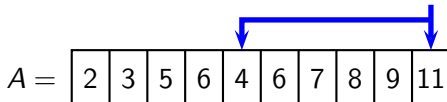
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

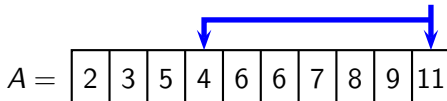
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

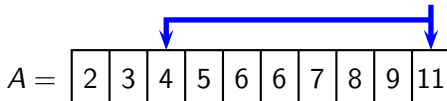
- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$



Insertion Sort

■ **Idea:** it is like sorting a hand of cards

- ▶ scan the sequence left to right
- ▶ pick the value at the current position a_j
- ▶ insert it in its correct position in the sequence $\langle a_1, a_2, \dots, a_{j-1} \rangle$ so as to maintain a sorted subsequence $\langle a_1, a_2, \dots, a_j \rangle$

$A =$

2	3	4	5	6	6	7	8	9	11
---	---	---	---	---	---	---	---	---	----

Insertion Sort (2)

INSERTION-SORT(A)

```
1  for  $i = 2$  to  $length(A)$   
2       $j = i$   
3      while  $j > 1$  and  $A[j - 1] > A[j]$   
4          swap  $A[j]$  and  $A[j - 1]$   
5           $j = j - 1$ 
```

Insertion Sort (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- Is INSERTION-SORT *correct*?
- What is the time complexity of INSERTION-SORT?
- Can we do better?

Complexity of INSERTION-SORT

INSERTION-SORT(A)

```
1  for  $i = 2$  to  $length(A)$ 
2       $j = i$ 
3      while  $j > 1$  and  $A[j - 1] > A[j]$ 
4          swap  $A[j]$  and  $A[j - 1]$ 
5           $j = j - 1$ 
```

Complexity of INSERTION-SORT

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- Outer loop (lines 1–5) runs exactly $n - 1$ times (with $n = \text{length}(A)$)
- What about the inner loop (lines 3–5)?
 - ▶ best and worst case?

Complexity of INSERTION-SORT (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

■ Best case:

Complexity of INSERTION-SORT (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- **Best case:** the inner loop is *never* executed
 - ▶ what case is this?

Complexity of INSERTION-SORT (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- **Best case:** the inner loop is *never* executed

- ▶ what case is this?

- **Worst case:**

Complexity of INSERTION-SORT (2)

```
INSERTION-SORT( $A$ )  
1  for  $i = 2$  to  $\text{length}(A)$   
2       $j = i$   
3      while  $j > 1$  and  $A[j - 1] > A[j]$   
4          swap  $A[j]$  and  $A[j - 1]$   
5           $j = j - 1$ 
```

- **Best case:** the inner loop is *never* executed
 - ▶ what case is this?
- **Worst case:** the inner loop is executed exactly $j - 1$ times for every iteration of the outer loop
 - ▶ what case is this?

Complexity of INSERTION-SORT (3)

- The worst-case complexity is when the inner loop is executed exactly $j - 1$ times, so

$$T(n) = \sum_{j=2}^n (j - 1)$$

Complexity of INSERTION-SORT (3)

- The worst-case complexity is when the inner loop is executed exactly $j - 1$ times, so

$$T(n) = \sum_{j=2}^n (j - 1)$$

$T(n)$ is the *arithmetic series* $\sum_{k=1}^{n-1} k$, so

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \Theta(n^2)$$

Complexity of INSERTION-SORT (3)

- The worst-case complexity is when the inner loop is executed exactly $j - 1$ times, so

$$T(n) = \sum_{j=2}^n (j - 1)$$

$T(n)$ is the *arithmetic series* $\sum_{k=1}^{n-1} k$, so

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \Theta(n^2)$$

- Best-case is $T(n) = \Theta(n)$

Complexity of INSERTION-SORT (3)

- The worst-case complexity is when the inner loop is executed exactly $j - 1$ times, so

$$T(n) = \sum_{j=2}^n (j - 1)$$

$T(n)$ is the *arithmetic series* $\sum_{k=1}^{n-1} k$, so

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \Theta(n^2)$$

- Best-case is $T(n) = \Theta(n)$
- Average-case is $T(n) = \Theta(n^2)$

Correctness

Correctness

- Does INSERTION-SORT terminate for all valid inputs?

Correctness

- Does INSERTION-SORT terminate for all valid inputs?
- If so, does it satisfy the conditions of the sorting problem?
 - ▶ A contains a *permutation* of the initial value of A
 - ▶ A is *sorted*: $A[1] \leq A[2] \leq \dots \leq A[\text{length}(A)]$

Correctness

- Does INSERTION-SORT terminate for all valid inputs?
- If so, does it satisfy the conditions of the sorting problem?
 - ▶ A contains a *permutation* of the initial value of A
 - ▶ A is *sorted*: $A[1] \leq A[2] \leq \dots \leq A[\text{length}(A)]$
- We want *a formal proof of correctness*
 - ▶ does not seem straightforward...

Loop Invariants

Loop Invariants

- We formulate a *loop-invariant* condition C
 - ▶ C must remain true *through* a loop

Loop Invariants

- We formulate a *loop-invariant* condition C
 - ▶ C must remain true *through* a loop
 - ▶ C is relevant to the problem definition: we use C at the end of a loop to prove the correctness of the result

Loop Invariants

- We formulate a *loop-invariant* condition C
 - ▶ C must remain true *through* a loop
 - ▶ C is relevant to the problem definition: we use C at the end of a loop to prove the correctness of the result
- Then, we only need to prove that the algorithm terminates

Loop Invariants (2)

Loop Invariants (2)

- Formulation: this is where we try to be smart
 - ▶ *the invariant must reflect the structure of the algorithm*
 - ▶ it must be the basis to prove the correctness of the solution

Loop Invariants (2)

- Formulation: this is where we try to be smart
 - ▶ *the invariant must reflect the structure of the algorithm*
 - ▶ it must be the basis to prove the correctness of the solution
- Proof of validity (i.e., that C is indeed a loop invariant): typical *proof by induction*
 - ▶ *initialization*: we must prove that *the invariant C is true before at the start of the first loop*
 - ▶ *maintenance*: we must prove that **if C is true at the beginning of a cycle then it remains true after one cycle**

Loop Invariant for INSERTION-SORT

INSERTION-SORT(A)

1 **for** $i = 2$ **to** $length(A)$

2 $j = i$

3 **while** $j > 1$ **and** $A[j - 1] > A[j]$

4 swap $A[j]$ and $A[j - 1]$

5 $j = j - 1$

Loop Invariant for INSERTION-SORT

```
INSERTION-SORT( $A$ )  
1  for  $i = 2$  to  $\text{length}(A)$   
2       $j = i$   
3      while  $j > 1$  and  $A[j - 1] > A[j]$   
4          swap  $A[j]$  and  $A[j - 1]$   
5           $j = j - 1$ 
```

- The main idea is to insert $A[i]$ in $A[1 \dots i - 1]$ so as to maintain a *sorted subsequence* $A[1 \dots i]$

Loop Invariant for INSERTION-SORT

```
INSERTION-SORT( $A$ )  
1  for  $i = 2$  to  $\text{length}(A)$   
2       $j = i$   
3      while  $j > 1$  and  $A[j - 1] > A[j]$   
4          swap  $A[j]$  and  $A[j - 1]$   
5           $j = j - 1$ 
```

- The main idea is to insert $A[i]$ in $A[1 \dots i - 1]$ so as to maintain a *sorted subsequence* $A[1 \dots i]$
- *Invariant:* (outer loop) the subarray $A[1 \dots i - 1]$ consists of the elements originally in $A[1 \dots i - 1]$ in sorted order

Loop Invariant for INSERTION-SORT (2)

INSERTION-SORT(A)

1 **for** $i = 2$ **to** $length(A)$

2 $j = i$

3 **while** $j > 1$ **and** $A[j - 1] > A[j]$

4 swap $A[j]$ and $A[j - 1]$

5 $j = j - 1$

Loop Invariant for INSERTION-SORT (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- **Initialization:** $i = 2$, so $A[1..i-1]$ is the single element $A[1]$

Loop Invariant for INSERTION-SORT (2)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- **Initialization:** $i = 2$, so $A[1..i-1]$ is the single element $A[1]$
 - ▶ $A[1]$ is trivially sorted

Loop Invariant for INSERTION-SORT (3)

INSERTION-SORT(A)

1 **for** $i = 2$ **to** $length(A)$

2 $j = i$

3 **while** $j > 1$ **and** $A[j - 1] > A[j]$

4 swap $A[j]$ and $A[j - 1]$

5 $j = j - 1$

Loop Invariant for INSERTION-SORT (3)

```
INSERTION-SORT(A)
1  for  $i = 2$  to  $\text{length}(A)$ 
2       $j = i$ 
3      while  $j > 1$  and  $A[j - 1] > A[j]$ 
4          swap  $A[j]$  and  $A[j - 1]$ 
5           $j = j - 1$ 
```

- **Maintenance:** At end of each iteration of the outer loop, The number that was at $A[i]$ *at the start* is inserted at its *correct position between $A[1 \dots i]$ at the end*

- ▶ k is the correct location of the number originally at index i
- ▶ shifts the subarray $A[k \dots i - 1]$ by one position to the right
- ▶ inserts key , which was originally in $A[i]$ at its proper position $1 \leq k \leq i - 1$, in sorted order

Loop Invariant for INSERTION-SORT (4)

INSERTION-SORT(A)

1 **for** $i = 2$ **to** $length(A)$

2 $j = i$

3 **while** $j > 1$ **and** $A[j - 1] > A[j]$

4 swap $A[j]$ and $A[j - 1]$

5 $j = j - 1$

Loop Invariant for INSERTION-SORT (4)

```
INSERTION-SORT(A)  
1  for i = 2 to length(A)  
2      j = i  
3      while j > 1 and A[j - 1] > A[j]  
4          swap A[j] and A[j - 1]  
5          j = j - 1
```

- **Termination:** INSERTION-SORT terminates with $i = \text{length}(A) + 1$; the invariant states that

Loop Invariant for INSERTION-SORT (4)

```
INSERTION-SORT( $A$ )  
1  for  $i = 2$  to  $\text{length}(A)$   
2       $j = i$   
3      while  $j > 1$  and  $A[j - 1] > A[j]$   
4          swap  $A[j]$  and  $A[j - 1]$   
5           $j = j - 1$ 
```

- **Termination:** INSERTION-SORT terminates with $i = \text{length}(A) + 1$; the invariant states that
 - ▶ $A[1 \dots i - 1]$ is a permutation of the original $A[1 \dots i - 1]$
 - ▶ $A[1 \dots i - 1]$ is sorted

Given the termination condition, $A[1 \dots i - 1]$ is the whole A

So INSERTION-SORT is *correct*!

Summary

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

1. Formulate an invariant C

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

1. Formulate an invariant C

2. **Initialization**

(for all valid inputs)

- ▶ prove that C holds right before the first execution of the first instruction of the loop

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

1. Formulate an invariant C

2. **Initialization** (for all valid inputs)

- ▶ prove that C holds right before the first execution of the first instruction of the loop

3. **Management** (for all valid inputs)

- ▶ prove that if C holds right before the first instruction of the loop, then it holds also at the end of the loop

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

1. Formulate an invariant C

2. **Initialization** (for all valid inputs)

- ▶ prove that C holds right before the first execution of the first instruction of the loop

3. **Management** (for all valid inputs)

- ▶ prove that if C holds right before the first instruction of the loop, then it holds also at the end of the loop

4. **Termination** (for all valid inputs)

- ▶ prove that the loop terminates, with some exit condition X

Summary

- You are given a problem P and an algorithm A
 - ▶ P formally defines a *correctness* condition
 - ▶ assume, for simplicity, that A consists of one loop
-

1. Formulate an invariant C

2. **Initialization** (for all valid inputs)

- ▶ prove that C holds right before the first execution of the first instruction of the loop

3. **Management** (for all valid inputs)

- ▶ prove that if C holds right before the first instruction of the loop, then it holds also at the end of the loop

4. **Termination** (for all valid inputs)

- ▶ prove that the loop terminates, with some exit condition X

5. Prove that $X \wedge C \Rightarrow P$, which means that A is correct